

Introduction

Questo documento (template del corso) descrive lo **Sprint 3**, che completa il sistema **cargoservice**. Percorso: Sprint 0 (requisiti e core business) → Sprint 1 (core eseguibile + sensor + timeout) → Sprint 2 (Out of service + display web-gui) → **Sprint 3** (questo): l'**IOPort** diventa una **web-gui interattiva** (pushbutton, display e **sensor con cui giocare**), **servita e aperta dall'applicazione**; il **marker** diventa un attore; il display mostra lo **stato reale** della stiva; è predisposto un **test automatizzato** (TP1).

Goal dello Sprint 3 (sottoinsieme del testo del committente): *"The display is used to show the answer to the request and to show the current state of the hold"* e *"a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed"*.
A valle della revisione dello Sprint 2 il **committente ha richiesto** che l'**IOPort** non sia una pagina di sola uscita ma una **vera interfaccia** (con **pushbutton**) **aperta dal sistema**, e un modo per **interagire col sensore** senza il dispositivo fisico.

Osservazioni del committente (revisione Sprint 2) e risposte di questo Sprint:

Osservazione	Risposta nello Sprint 3
la web-gui ridotta a pagina HTML di sola uscita non basta : manca il bottone e l' applicazione (o un servizio) deve aprire la pagina	l'IOPort è ora una pagina interattiva (ioport.html) con pushbutton , display e sensore, servita e aperta dall'app (WebIoPort, server HTTP interno)
discutere e giustificare la web-gui nel PDF (funzioni, tecnologia, perché)	sezione <i>"Web-GUI dell'IOPort"</i> nel Problem analysis: funzioni, scelta di MQTT/websocket, perché servita dall'app, trasparenza del dispositivo
fornire un' applicazione che faccia da sensore , non hard-coded e senza la Raspberry, con cui giocare	il sensore è uno slider (+ bottoni rapidi) nella pagina: emette distance(D) in tempo reale; rimossi gli attori iosensor/ pushbutton hard-coded

Sintesi dello Sprint precedente (punto di partenza)

Lo Sprint 2 è **eseguito e testato**: Out of service ($D > D_{FREE}$ per $\sim 3s$ → display "Out of service", richieste → `retrylater`), rientro in servizio, display **web-gui** (MQTT/websocket), `sensormonitor` dal dato grezzo `distance(D)` in formato PicoW reale. Robustezza verificata: `moverobotfailed` → slot liberato, servizio disponibile. Documento: [Sprint2_v1](#).

Requirements

Requisiti oggetto di questo Sprint (testo esatto in [Sprint 0 – Requirements](#)):

- *"The user makes a request to load a container **by pushing a button** of an **IOPort** device, that also has a **display**."*
- *"The **display** is used to **show the answer to the request** and to show the **current state of the hold**."*
- *"During temporary storage, a 'marker' device **labels the container with an identification barcode** and **signals when this marking activity is completed**."*

Più la richiesta esplicita del committente alla revisione: l'IOPort va realizzato come **interfaccia con pulsante, aperta dal sistema**, e dev'essere possibile **interagire col sensore** (sonar) senza l'hardware.

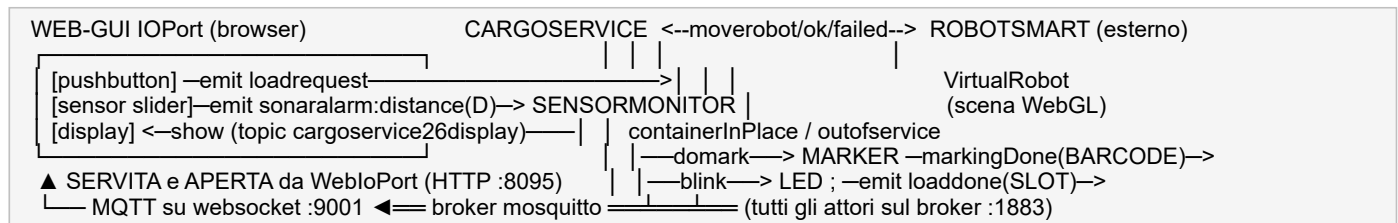
Requirement analysis

Termine	Analisi
IOPort (pushbutton + display)	è un boundary del sistema: un dispositivo di confine con cui l'utente interagisce. Identificato nell'analisi DDR come confine input (pushbutton, sonar) e output (display). La sua realizzazione naturale è una GUI : in questo progetto una pagina web
"push a button"	l'utente genera un evento : il pushbutton non attende un valore di ritorno sincrono, notifica la volontà di caricare → loadrequest modellato come EVENT (non request/reply)
"show the answer to the request"	la risposta (reserved/retrylater/ reject) è comunicata sul display , in modo asincrono , esattamente come un pannello pulsante+display reale. Quindi la "answer" è uno stato mostrato, non un reply di protocollo
sensor (sonar)	misura la distanza del container nell'area dell'IOPort: dato grezzo distance(D) . Per "giocarci" senza PicoW serve un generatore interattivo dello stesso dato (uno slider)
marker che "segnala"	il completamento della marcatura è una notifica asincrona di un dispositivo con un suo tempo proprio: natura da attore ; il cargoservice attende il segnale , non un tempo fisso
barcode	identificativo assegnato durante la marcatura: è un dato della notifica (markingDone(BARCODE)); viene memorizzato nello slot (Hold.confirmStored(slot,barcode)) e mostrato sul display (es. slot1=PIENO(bc1))
stato corrente della stiva	derivabile dal POJO Hold: stateDescription() (quali slot pieni/liberi + barcode) mostrato a fine ciclo
notifiche stale	un containerInPlace rimasto in coda senza richiesta attiva non deve avviare trasporti: va scartato

Ruolo dell'analisi. La web-gui **non è una scelta estetica**: è la **realizzazione del boundary IOPort** individuato nell'analisi. La mappatura è 1:1 — pushbutton (confine di input) → bottone; sonar (confine di input) → slider; display (confine di output) → area di testo. Il design **segue l'analisi**: prima si è capito *cosa* è l'IOPort (un confine con due ingressi e un'uscita), poi si è scelto *come* realizzarlo (pagina web su MQTT).

Problem analysis

Architettura finale del sistema



Scelta	Motivazione
IOPort = web-gui (pushbutton+display+sensor) servita dall'app	realizza il boundary individuato in analisi; un'unica pagina raccoglie i tre confini del dispositivo; l'app la eroga e la apre (come un device che "va online")
loadrequest come EVENT (non request/reply)	il pushbutton notifica ; la risposta è sul display (requisito), non un reply sincrono: modello più fedele a un pannello pulsante+display reale
sensor = slider che emette distance(D)	l'utente " diventa il sonar ": niente PicoW, niente codice hard-coded; stesso dato grezzo del dispositivo reale
marker attore con domark/markingDone (dispatch)	il requisito chiede un dispositivo che segnala : il cargoservice resta in marking finché arriva markingDone(BARCODE); il barcode viaggia nella notifica
Event loaddone(SLOT)	notifica di fine carico: chiude il ciclo verso display/tester senza accoppiare il cargoservice al chiamante
dropStale in available	un containerInPlace senza richiesta attiva viene consumato e scartato (osservato nei test dello Sprint 2)

Web-GUI dell'IOPort — scelta ingegneristica

L'IOPort è il **boundary** tra l'utente e il servizio. Realizzarlo come **pagina web** è una decisione di progetto che va giustificata su tre piani: **funzioni**, **tecnologia** e **responsabilità** (chi serve la pagina).

Elemento	Funzione	Messaggio (MQTT)
Pushbutton	l'utente chiede di caricare un container	emette <code>msg(loadrequest,event,webioport,none,loadrequest(customer),N)</code>
Sensor (slider 0–100 cm + bottoni rapidi)	l'utente "muove il container" / simula un guasto del sonar	emette <code>distance(D)</code> a ~3 campioni/s (come il sonar reale)
Display	mostra la risposta (reserved/retrylater/reject), gli stati (engaged, Out of service, serviceworking) e lo stato della stiva	sottoscrive il topic <code>cargoservice26display</code>

Perché MQTT su websocket (e non un endpoint HTTP dedicato). La pagina parla lo **stesso protocollo** del resto del sistema: i messaggi hanno il formato `msg(ID,event,SENDER,RECEIVER,CONTENT,SEQ)`, identico a quello pubblicato dal **PicoW reale** (`Picow/Sonar/sonarMqtt.py`). Conseguenza ingegneristica chiave: la web-gui è **solo un altro partecipante MQTT**; il **dispositivo fisico** (sonar PicoW, pulsante hardware) può **sostituire** lo slider/bottone **senza modificare una riga** del `cargoservice`. Il sistema dipende dall'**informazione scambiata**, non dal dispositivo (*trasparenza del dispositivo* / basso accoppiamento). Il websocket (porta 9001) è ciò che permette al **browser** di parlare lo stesso bus MQTT degli attori.

Canali separati (pushbutton vs sensor). Il pushbutton e il sensor usano **topic MQTT distinti**: `cargoservice26in_out` (lo ascolta solo il `cargoservice`) e `cargoservice26sonar` (lo ascolta solo il `sensormonitor`). Scelta motivata dal **modello di esecuzione del runtime**: se i due eventi condividessero un unico topic, ogni messaggio verrebbe consegnato e **parsato in parallelo** da due attori, e il parser del runtime (tuProlog) **non è thread-safe** → sotto la **raffica** di letture del sensor la connessione MQTT cade. Separare i canali garantisce che **ogni messaggio sia parsato da un solo attore**: è una decisione di progetto guidata dai vincoli reali della piattaforma (osservata e corretta in fase di test).

Perché la pagina è SERVITA e APERTA dall'applicazione. Alla revisione il committente ha giustamente osservato che una pagina aperta a mano (`file://`) non è un dispositivo del sistema. Qui un piccolo **server HTTP interno** (`WebIoPort`, classe `com.sun.net.httpserver.HttpServer` — **già nel JDK, nessuna dipendenza nuova**) eroga `ioport.html` su `http://localhost:8095`, e il suo costruttore **apre il browser** all'avvio. L'IOPort quindi **"va online" col sistema**, come farebbe un device reale: è il sistema a renderlo disponibile, non l'operatore. È il pattern delle web-gui del corso (*conway26GuiAlone* con Javalin), qui ottenuto **senza aggiungere Javalin**.

Perché un POJO (withobj) e non un QActor. L'IOPort è un **dispositivo di confine**, non un attore logico del dominio. Coerentemente col pattern del corso (`devices.*`), i device sono POJO iniettati nel QActor con `withobj` (`withobj webio using "devices.WebIoPort()"`): l'interattività vive nel **browser**, il modello a attori resta pulito e si occupa solo della **logica** del servizio.

Perché il web e non una GUI desktop. Portabilità (qualunque browser, nessun runtime aggiuntivo), **decoupling** via MQTT (la GUI e il servizio non si conoscono), aderenza al pattern del corso e possibilità di **osservare/comandare anche da remoto**. La pagina è volutamente **thin**: nessuna logica di dominio, solo pubblicazione di eventi e visualizzazione di stato.

Piano di lavoro (previsione, ore/uomo)

Attività	Previsione	Effettivo
analisi (IOPort boundary, marker/barcode, stale)	1.5 h	~1.5 h
web-gui interattiva (<code>ioport.html</code> + <code>WebIoPort</code>) + modello qak	3.5 h	~4 h
test end-to-end + messa a punto della demo	2.5 h	~3 h (vedi scostamento)

documento + slides + preparazione colloquio	2 h	~2 h
totale	9.5 h	~10.5 h

Scostamento: +1 h, dovuto a (a) la messa a punto della web-gui interattiva e del server che la apre, e (b) l'ambiente robotico (la scena WebGL del VirtualRobot deve restare in primo piano: in background il browser rallenta i frame e gli step possono fallire). Documentato nel Deployment.

Distribuzione dei compiti: progetto **individuale** (Alexander Kreuzer).

Diario di bordo dello Sprint

Data	Attività
10/06	marker come attore (domark/markingDone+barcode); stato reale della stiva sul display (Hold.stateDescription)
11/06	tester TP1 automatizzato (loaddone); dropStale; test end-to-end
12/06	revisione del committente: IOPort → web-gui interattiva (pushbutton + sensor giocabile) servita e aperta dall'app (WebIoPort); loadrequest → EVENT; rimossi iosensor/pushbutton hard-coded; documento e slides aggiornati
13–17/06	test end-to-end del carico di TUTTA la stiva: scoperta una incoerenza coordinate↔mappa del DDR (vedi sotto) — slot3=(3,2) e slot4=(5,3) cadono su celle-ostacolo della tf25map (pianificatore A* → piano vuoto). Corrette su celle raggiungibili ; verificate le 4 cariche (stiva piena). Fix di affidabilità della scena (ricreare il container, non solo restart)

Incoerenza coordinate ↔ mappa del DDR (rilevata col testing, da confermare col committente). Le posizioni degli slot provengono dai comandi del docente (Robotsmart26Cmds: (1,1)(1,4)(3,2)(5,3)(4,3)(3,4)). Verificandole sulla mappa built-in del robot (robotsmart26/tf25map.txt, X=riga Y=colonna) col **pianificatore A* del robot**: solo (1,1)(1,4)(4,3)(3,4) sono **raggiungibili** (cella libera); (3,2) e (5,3) cadono su un **ostacolo / sul muro** e l'A* restituisce **piano vuoto** (irraggiungibili). Perciò slot3 e slot4 (già annotati come *provvisori* in Hold.java) sono stati portati su celle libere — slot3 → (4,3) (coordinata del docente, move43) e slot4 → (4,2) (derivata dalla mappa, da confermare). Verifica diretta col robot: buildPlan a (4,3) = "lwrwwwlww", a (4,2) = "lwrwwwlw" (raggiungibili); a (3,2) e (5,3) = " (irraggiungibili). Con la correzione le **4 cariche** vanno a buon fine (slot1..slot4 = PIENO).

Test plans

TP1 CARICO NOMINALE (AUTOMATIZZATO, demo finale)
 il TESTER emette loadrequest + sonaralarm:distance(20) -> interroga getholdstate
 PASS sse uno slot risulta (pieno) -> stampa "*** TP1 PASS ***"
 TP5 OUT OF SERVICE (regressione Sprint2): slider D>DFREE ~3s -> "Out of service"; pushbutton -> retrylater
 TP6 RIENTRO (regressione Sprint2): slider D rientra ~3s -> "serviceworking"; pushbutton -> reserved
 TP7 MARCATURA (Sprint3): in slot5 il marker assegna il barcode (bcN) e segnala markingDone
 PASS sse il trasporto prosegue SOLO dopo markingDone e il barcode è nel log
 TP8 NOTIFICA STALE (Sprint3): containerInPlace senza richiesta attiva -> scartato
 PASS sse il sistema resta available (nessun trasporto spontaneo)
 TP-GUI IOPort INTERATTIVO (Sprint3): l'app apre la pagina; il pushbutton avvia il carico;
 lo slider del sensore pilota presenza/Out-of-service; il display riflette risposta e stato stiva

Project

Sottosistema eseguibile finale: [TemaFinale26/Sprint3/cargoservice26](#) — modello [cargoservice26.gak](#):

Componente	Realizzazione
cargoservice	FSM: evalRequest (whenEvent loadrequest), marking/markingsCompleted (attesa markingDone), dropStale, emit loaddone(SLOT), display con hold.stateDescription(); withobj webio apre la web-gui
web-gui IOPort (nuova)	ioport.html : pushbutton + sensor (slider) + display, su MQTT/websocket (:9001)

WebIoPort (nuovo)	WebIoPort.java : server HTTP (JDK) che serve e apre la pagina su :8095 (nessuna dipendenza nuova)
marker	Marker.kt : riceve domark(SLOT), marca (1s, simulato), risponde markingDone(bcN)
tester (TP1)	Tester.kt : emette loadrequest+sonaralarm e verifica lo stato; esito PASS/FAIL a video (config cargoservice26tp1.pl, gradlew runTp1)
sensormonitor	come Sprint 2: dal grezzo distance(D) (slider o PicoW) a containerInPlace/outofservice
DisplayMqtt	pubblica lo stato (retained) su cargoservice26display; autodetect dell'IP del sito (broker Docker)

Due configurazioni: cargoservice26.pl (demo **interattiva**: cargoservice + sensormonitor + marker, l'utente usa la web-gui) e cargoservice26tp1.pl (prova **automatizzata** TP1, aggiunge il tester).

Testing

POJO: TestHold = **28 PASS, 0 FAIL** (con stateDescription, stateTerm, **barcode memorizzato e coordinate corrette**). **Build:** gradlew build = BUILD SUCCESSFUL. **Demo ESEGUITA** (output reale, 13/06; robotsmart + VirtualRobot + mosquitto attivi): gli eventi del **pushbutton** e del **sensor** della web-gui pilotano il ciclo completo (il robot esegue il trasporto IOport → slot5 → slot1 → HOME):

```
[DISPLAY] reserved slot1          <- pushbutton (loadrequest event) -> riserva
[LED] blink ON (engaged)
cargoservice | engaged: attendo il container nell'area del sensor (timeout 30s)
sensormonitor | presenza container (D<DFREE/2 per ~3s) -> containerInPlace <- sensor (slider)
cargoservice | chiedo la marcatura in slot5 ...
marker      | marcatura in corso, barcode=bc1          [TP7]
cargoservice | marcatura completata, barcode=bc1
[DISPLAY] hold: slot1=PIENO slot2=libero slot3=libero slot4=libero      [stato reale]
cargoservice | richiesta completata: container in slot1
cargoservice | emit loaddone on cargoservice26in_out
cargoservice | available (attende loadrequest)
```

Demo interattiva (TP-GUI, eseguita): all'avvio del cargoservice la pagina `ioport.html` si apre da sola (WebIoPort); premendo il **pushbutton** parte il carico, e muovendo lo **slider** del sensore si pilotano presenza container (D<30 per ~3s), **Out of service** (D>60 per ~3s) e rientro: il **display** riflette risposta e stato della stiva in tempo reale. La prova **TP1 automatizzata** (gradlew runTp1) ottiene la stessa sequenza con l'attore **tester** al posto della web-gui, stampando ***** TP1 PASS *****. Verificati inoltre **TP4** (nessuna presenza entro 30s → disengage) e **TP8** (notifica stale scartata).

Deployment

```
(da TemaFinale26/Sprint3/ — oppure: powershell -ExecutionPolicy Bypass -File demo.ps1)
1) docker compose -f ../../robotmart26/yamls/unibobasic26.yaml up (VirtualRobot + GUI + mosquitto)
   + abilitare il listener websockets :9001 di mosquitto (lo fa demo.ps1)
2) aprire http://localhost:8090 e TENERLA IN PRIMO PIANO (la scena WebGL ESEGUE i movimenti)
3) cd ../../robotmart26 ; ./gradlew run (servizio del docente, 8020)
4) cd cargoservice26 ; ./gradlew run (l'app APRE da sola la web-gui :8095)
5) usare la web-gui: premere il pushbutton, muovere lo slider del sensore
   (per la prova automatizzata: ./gradlew runTp1)
```

Nota infrastruttura. Su questa macchina convive un mosquitto nativo di Windows su `127.0.0.1:1883`: i publisher Java (DisplayMqtt) usano l'**IP del sito** per raggiungere il mosquitto **Docker** (dove c'è il websocket :9001 della pagina). Il fix definitivo è disabilitare il servizio nativo. Dettagli e script in `demo.ps1`.

Maintenance — sintesi finale del sistema

Copertura dei requisiti del committente (architettura finale e dove ogni requisito è soddisfatto):

Requisito	Dove	Sprint
richiesta col pushbutton dell'IOPort	web-gui (bottone) → EVENT loadrequest	3
risposta reserved/retrylater/reject sul display	FSM evalRequest + display.show() (web-gui)	0–1, 3
LED lampeggiante mentre engaged	led.blink (indicatore di stato)	0–1
container nell'area del sensor entro ~30s, altrimenti disengage	waitContainer (whenTime 30000) + sensormonitor	1–2
trasporto IOPort → slot5 → slot riservato (cargorobot)	client di robotsmart (moverobot)	0–1
marcatore con barcode e segnale di completamento	attore marker (markingDone(bcN))	3
retrylater se IOPort occupato o Out of service	evalRequest / stato outOfService	0, 2
reject a stiva piena	Hold.isFull()	0–1
display: risposta, stato stiva , Service working / Out of service	web-gui MQTT + stateDescription()	2–3
Out of service se D>DFREE per ≥ 3 s (guasto sonar)	sensormonitor (finestra 3 campioni)	2
interagire col sensore senza hardware	slider della web-gui → distance(D)	3

I dispositivi (sonar, pushbutton, marker, display, LED) sono **simulati ma sostituibili**: il cargoservice dipende solo dall'informazione scambiata (il PicoW reale pubblica già lo stesso distance(D) della web-gui). DFREE, finestra 3s, posizioni degli slot: **parametri** da confermare col committente.

Consegna (§32.2): questa fase viene pubblicata sul sito **48h prima del colloquio** con mail al docente; demo live 10–15 min con **web-gui interattiva** e **TP1 automatizzato**; slides (<10) sul processo e sull'architettura finale.



By Alexander Kreuzer

Email: Alexander.kreuzer@studio.unibo.it

GIT repo: https://github.com/Alexing-uni/ISS26_Alexander_Kreuzer